

Movie Recommendation System on AWS

Name: Riya Kalokhe Roll No.:TE29

1. Project Overview

This project is a content-based movie recommendation system built entirely on AWS cloud services. It uses the TMDB 5000 Movies dataset to recommend movies based on similar genres and keywords, queried using pure SQL through Amazon Athena without any machine learning model.

The system takes a movie as input and returns the top 10 most similar movies ranked by a similarity score calculated from matching genres and keywords.

2. AWS Architecture & Services Used

2.1 Amazon S3 - Storage Layer

Amazon S3 (Simple Storage Service) is the storage foundation of the project. The main bucket movie-reco-riya-18 was created in the Asia Pacific (Mumbai) region and contains all project data organized in folders.

Folder / File	Type	Purpose
raw/	Folder	Original CSV file (tmdb_5000_movies.csv)
processed/	Folder	Intended for Glue ETL output (empty)
Unsaved/	Folder	Athena query result .txt files (caused errors)
run-177XXXX.snappy	Parquet files	Glue ETL output accidentally at bucket root

Screenshot: S3 Buckets Overview

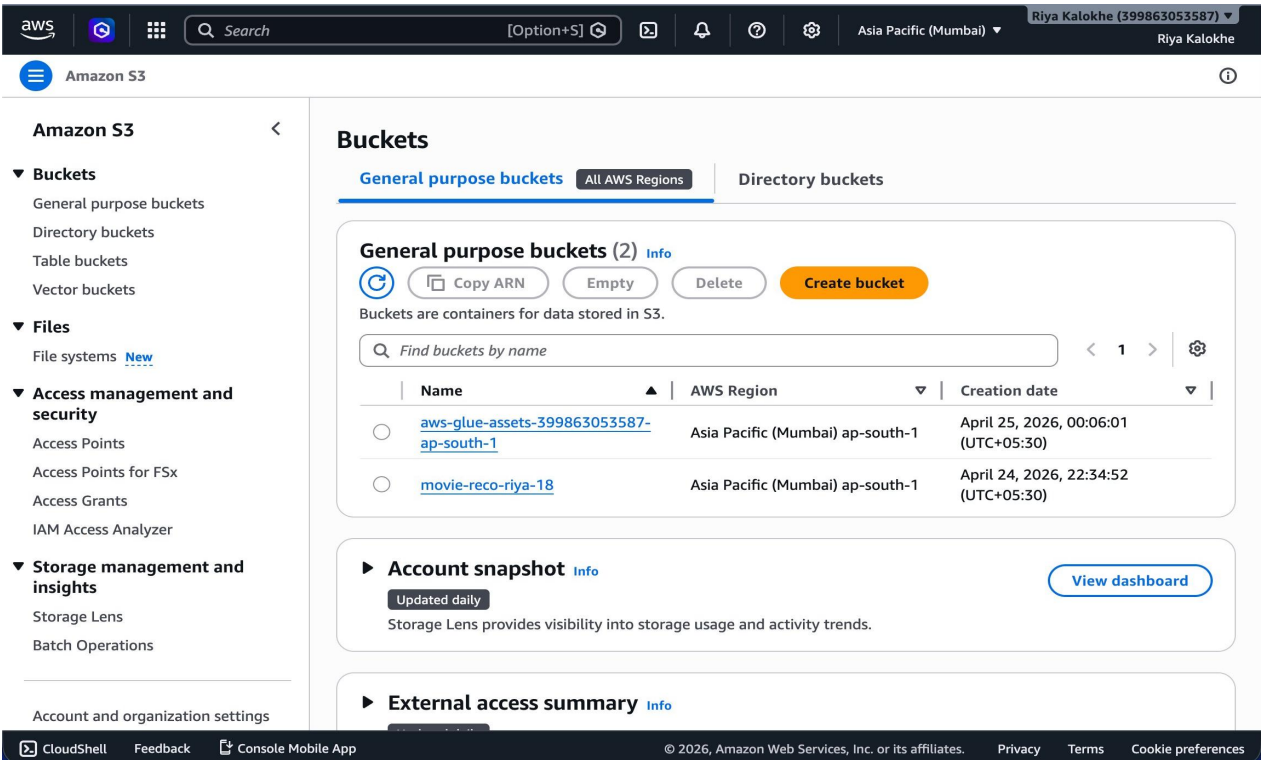


Figure 1: Amazon S3 console showing 2 buckets - aws-glue-assets and movie-reco-riya-18

Screenshot: S3 Bucket Contents

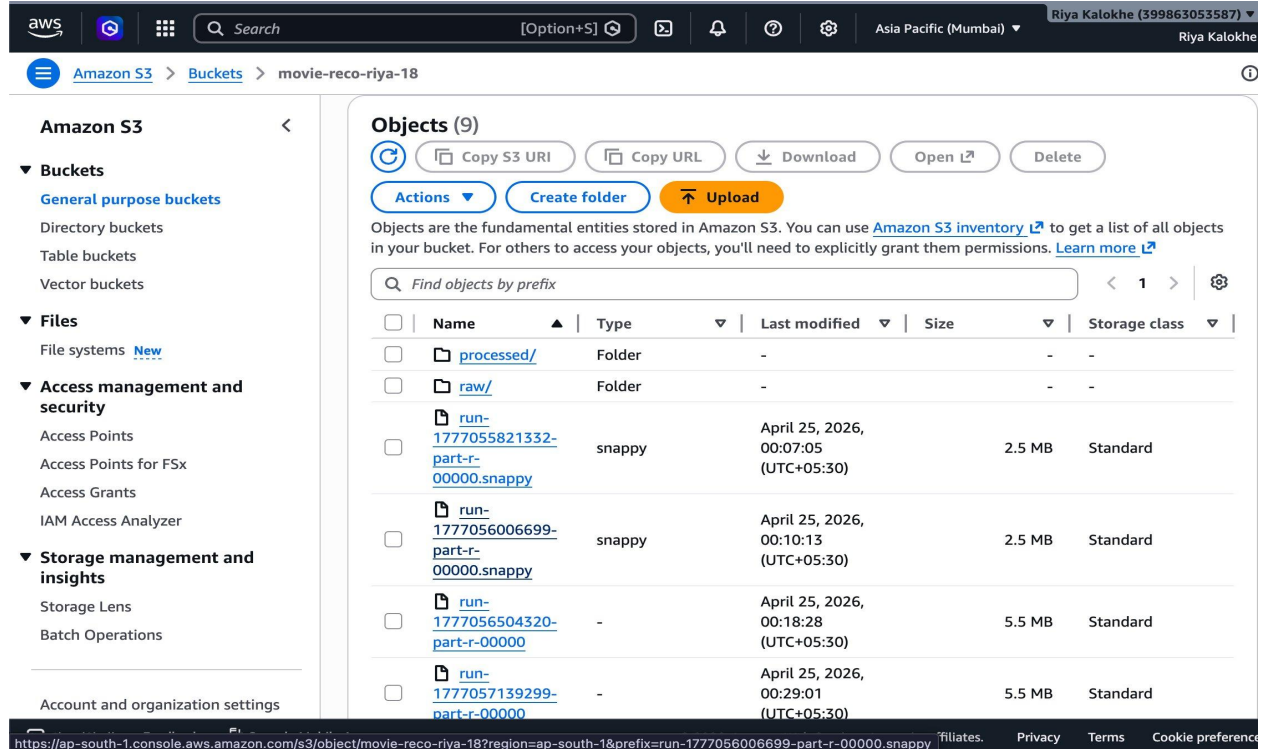


Figure 2: Inside movie-reco-riya-18 bucket showing folders and .snappy Glue output files at root

2.2 AWS Glue - Data Processing Layer

AWS Glue was used to build a Visual ETL (Extract, Transform, Load) pipeline called movie-job. The pipeline reads from the AWS Glue Data Catalog and writes processed output to Amazon S3. It also automatically cataloged the data schema with proper column names and data types.

The Glue ETL job created the following components:

- Data source: AWS Glue Data Catalog
- Data target: Amazon S3 bucket
- Auto-detected schema with 21 columns including genres and keywords as array types
- Output files in Snappy Parquet format

Screenshot: AWS Glue Visual ETL Job

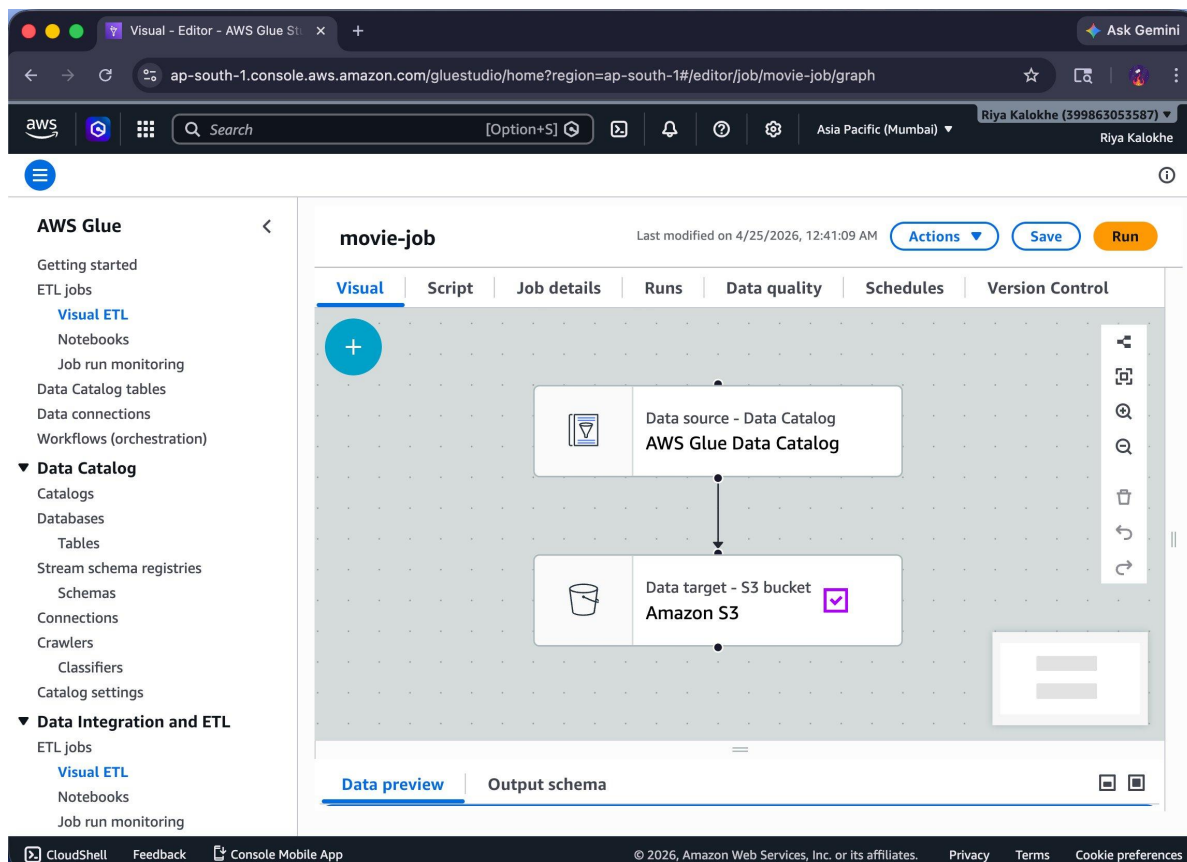


Figure 3: Glue Visual ETL job (movie-job) showing Data Catalog source to S3 target pipeline

Screenshot: Glue Data Catalog Table Schema

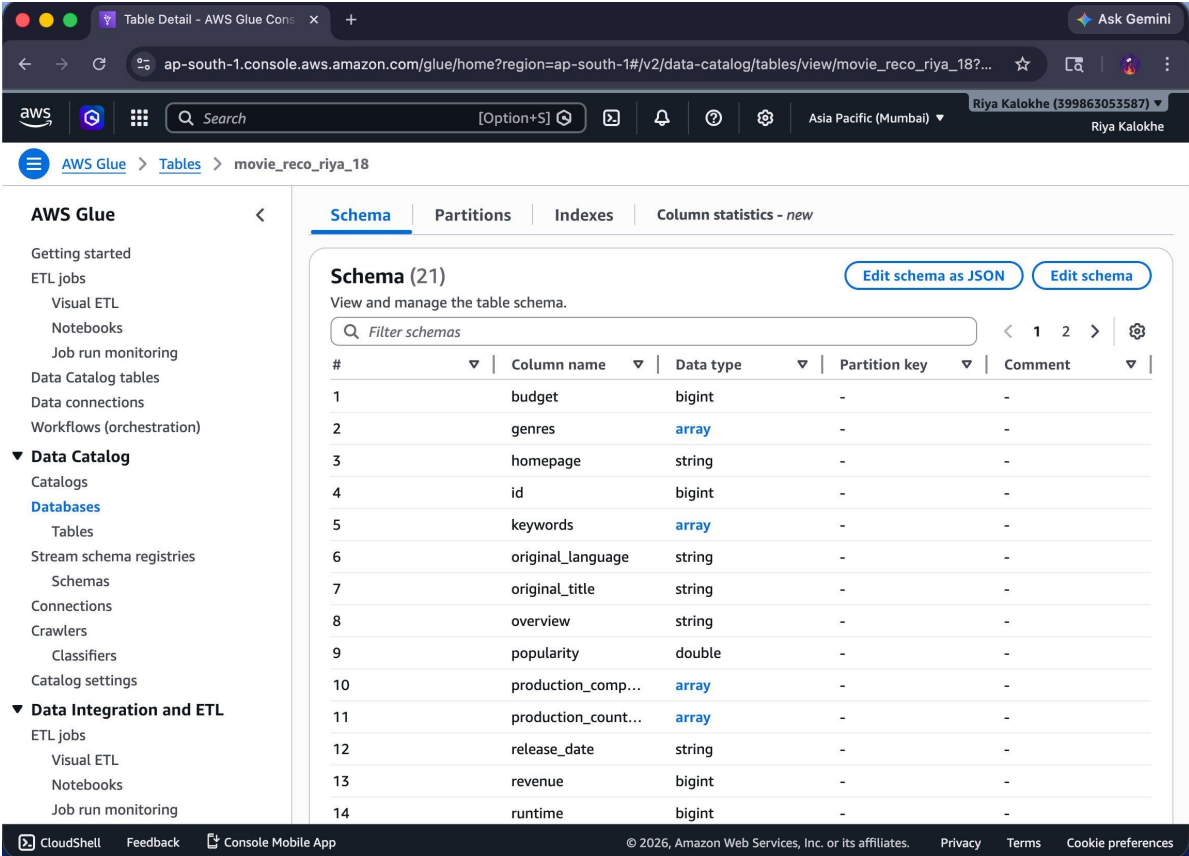


Figure 4: Properly typed schema with 21 columns - genres and keywords as array, popularity as double

2.3 AWS IAM - Security Layer

AWS Identity and Access Management (IAM) was used to manage permissions. The GlueS3Role was created to give the Glue ETL job permission to read from and write to S3. Without this role, Glue would not be able to access the movie data.

Screenshot: IAM Roles

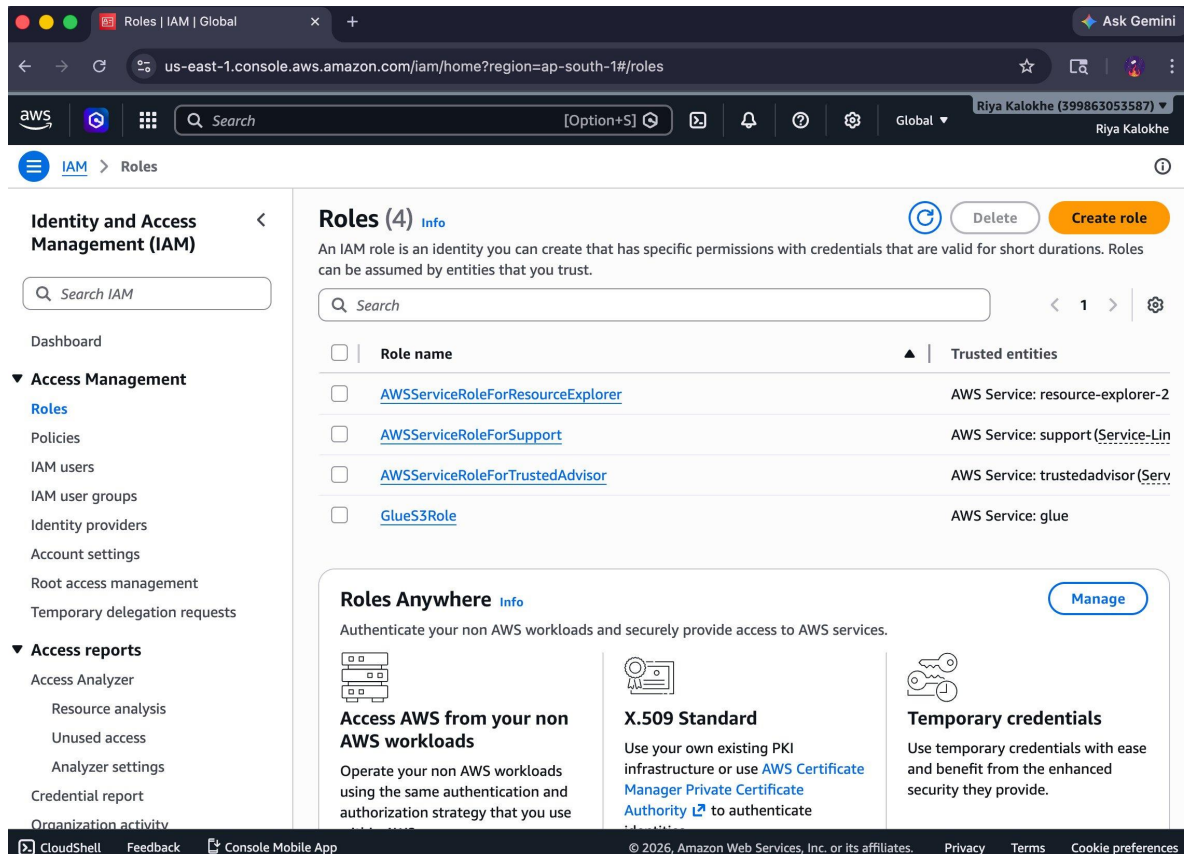


Figure 5: IAM Roles showing GlueS3Role that grants Glue access to S3

2.4 Amazon Athena - Query Layer

Amazon Athena is a serverless interactive query service used to run SQL queries directly on the CSV data in S3. It was the primary interface for building and testing the recommendation logic.

Key Athena configurations used:

- Data source: AwsDataCatalog
- Database: movies_db
- SerDe: OpenCSVSerde to handle JSON embedded inside CSV fields
- Table: movies_data pointing to s3://movie-reco-riya-18/raw/

Description

Screenshot: Broken Column Parsing in Athena

The screenshot shows the Amazon Athena Query Editor interface. The query has completed successfully, with a run time of 523 ms and 3.43 MB of data scanned. The results table shows 10 rows of data. The columns are: #, budget, genres, homepage, id, and keywords. The 'genres' column contains JSON fragments like [{"id": 28}, {"id": 12}, {"id": 28}, {"id": 28}, {"id": 28}, {"id": 14}, {"id": 16}, {"id": 28}, {"id": 12}. The 'homepage' column contains JSON fragments like {"name": "Action"}, {"name": "Adventure"}, {"name": "Action"}, {"name": "Action"}, {"name": "Action"}, {"name": "Fantasy"}, {"name": "Animation"}, {"name": "Action"}, {"name": "Adventure"}. The 'keywords' column contains JSON fragments like {"name": "}, {"name": "}, {"name": "}, {"name": "}, {"name": "}, {"name": "}, {"name": "}, {"name": "}, {"name": "}. The misalignment is evident as the data in the 'genres' and 'homepage' columns is shifted to the right, causing the 'id' and 'keywords' columns to be misaligned.

#	budget	genres	homepage	id	keywords
1	237000000	[{"id": 28}	{"name": "Action"}		{"name": "
2	300000000	[{"id": 12}	{"name": "Adventure"}		{"name": "
3	245000000	[{"id": 28}	{"name": "Action"}		{"name": "
4	250000000	[{"id": 28}	{"name": "Action"}		{"name": "
5	260000000	[{"id": 28}	{"name": "Action"}		{"name": "
6	258000000	[{"id": 14}	{"name": "Fantasy"}		{"name": "
7	260000000	[{"id": 16}	{"name": "Animation"}		{"name": "
8	280000000	[{"id": 28}	{"name": "Action"}		{"name": "
9	250000000	[{"id": 12}	{"name": "Adventure"}		{"name": "

Figure 6: Athena query showing misaligned columns - genres column showing JSON fragments due to default CSV SerDe

This screenshot revealed the core problem. The default CSV SerDe could not handle JSON embedded inside CSV fields, causing all columns to shift and misalign. This was fixed by switching to OpenCSVSerde.

Screenshot: Misaligned Data with Scientific Notation

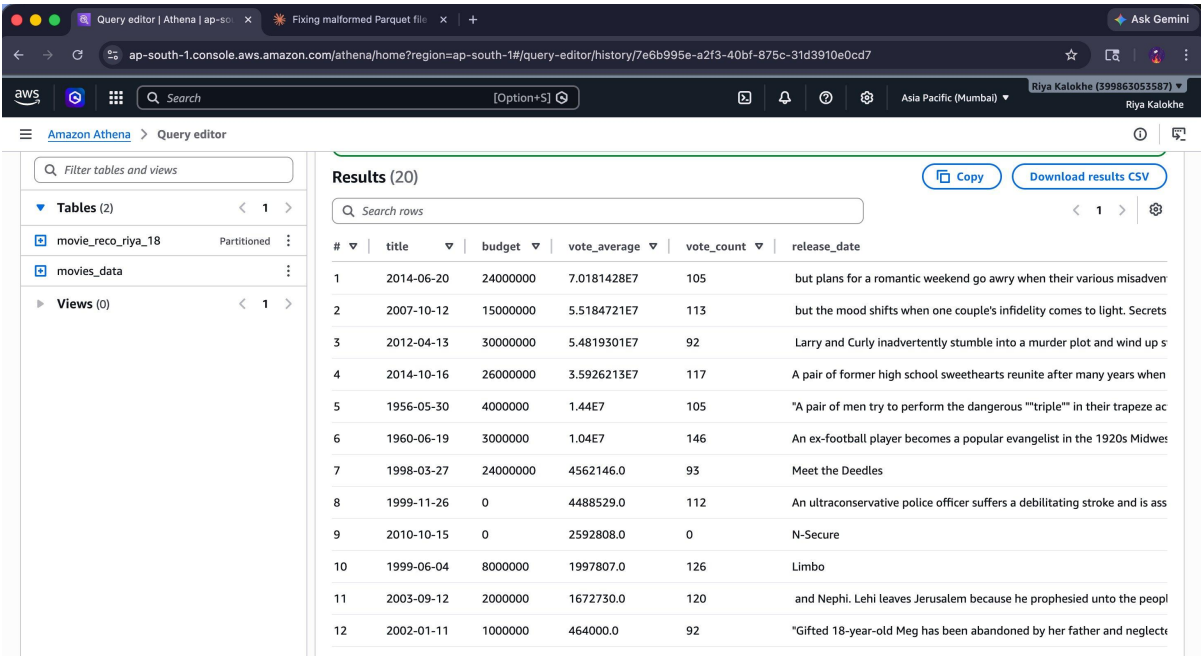


Figure 7: Title column showing dates, vote_average in scientific notation - columns clearly in wrong positions

Screenshot: Wrong Table Query Results

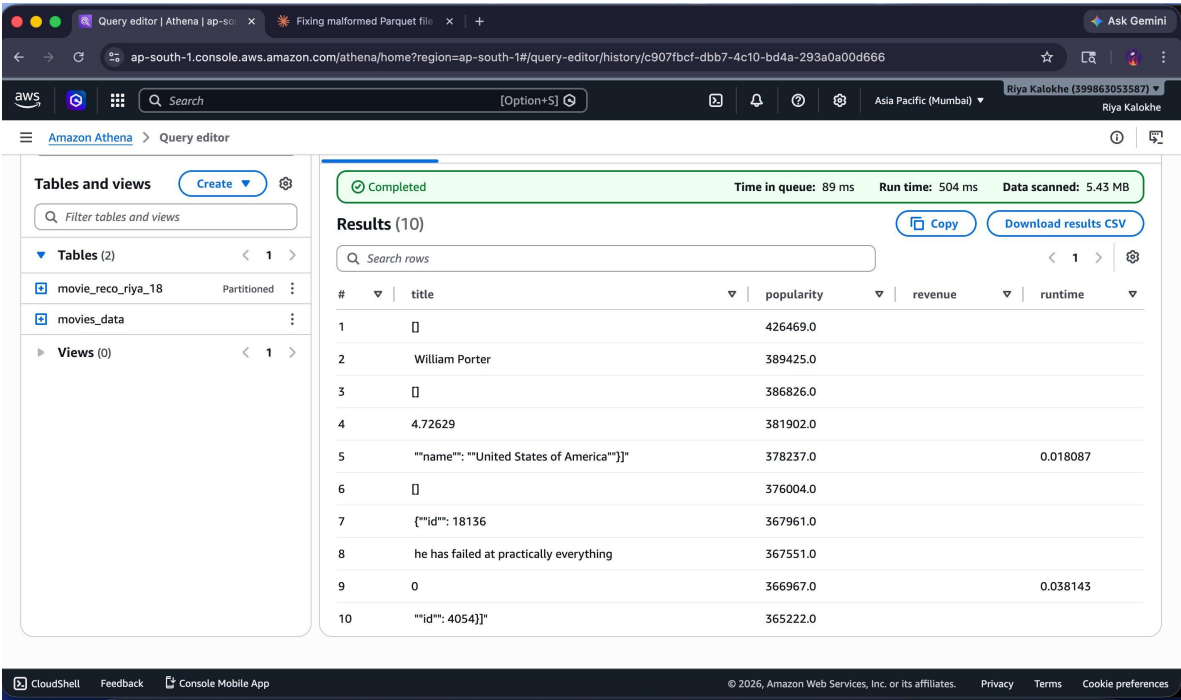


Figure 8: Results showing garbage in title column - [], William Porter, JSON fragments instead of movie titles

This screenshot shows what happened when the wrong Glue Parquet table (movie_reco_riya_18) was queried instead of the fixed CSV table (movies_data). The Parquet files at the bucket root had corrupted data.

Screenshot: Recommendation SQL Query

Query 10 : X Query 12 : X Query 13 : X Query 14 : X Query 15 : X Query 16 : X > (+)

```
1 SELECT
2   title,
3   vote_average,
4   release_date,
5   (
6     CASE WHEN genres LIKE '%Action%' THEN 1 ELSE 0 END +
7     CASE WHEN genres LIKE '%Adventure%' THEN 1 ELSE 0 END +
8     CASE WHEN genres LIKE '%Fantasy%' THEN 1 ELSE 0 END +
9     CASE WHEN genres LIKE '%Science Fiction%' THEN 1 ELSE 0 END +
10    CASE WHEN keywords LIKE '%space%' THEN 1 ELSE 0 END +
11    CASE WHEN keywords LIKE '%alien%' THEN 1 ELSE 0 END +
12    CASE WHEN keywords LIKE '%futuristic%' THEN 1 ELSE 0 END +
13    CASE WHEN keywords LIKE '%battle%' THEN 1 ELSE 0 END +
14    CASE WHEN keywords LIKE '%soldier%' THEN 1 ELSE 0 END
15  ) AS similarity_score
16 FROM movies_data
17 WHERE title != 'Avatar'
18 HAVING (
19   CASE WHEN genres LIKE '%Action%' THEN 1 ELSE 0 END +
20   CASE WHEN genres LIKE '%Adventure%' THEN 1 ELSE 0 END +
21   CASE WHEN genres LIKE '%Fantasy%' THEN 1 ELSE 0 END +
22   CASE WHEN genres LIKE '%Science Fiction%' THEN 1 ELSE 0 END +
23   CASE WHEN keywords LIKE '%space%' THEN 1 ELSE 0 END +
24   CASE WHEN keywords LIKE '%alien%' THEN 1 ELSE 0 END +
25   CASE WHEN keywords LIKE '%futuristic%' THEN 1 ELSE 0 END +
26   CASE WHEN keywords LIKE '%battle%' THEN 1 ELSE 0 END +
27   CASE WHEN keywords LIKE '%soldier%' THEN 1 ELSE 0 END
28 ) >= 5
29
```

SQL Ln 30, Col 10

Figure 9: The core recommendation SQL using CASE WHEN LIKE scoring for genres and keywords

Screenshot: Final Working Recommendations for Avatar

Completed Time in queue: 76 ms Run time: 915 ms Data scanned: 5.43 MB

Results (10) Copy Download results CSV

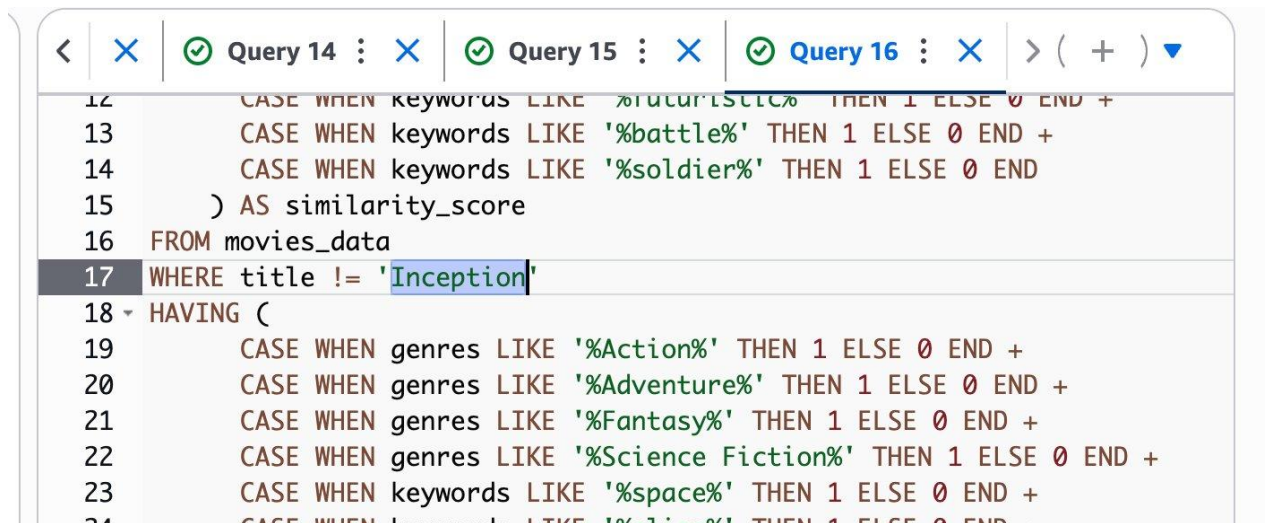
Search rows

#	title	vote_average	release_date	similarity_score
1	Star Trek Into Darkness	7.4	2013-05-05	6
2	The Fifth Element	7.3	1997-05-07	6
3	Independence Day	6.7	1996-06-25	6
4	Starship Troopers	6.7	1997-11-06	6
5	Ender's Game	6.6	2013-10-23	6
6	Final Fantasy: The Spirits Within	5.9	2001-07-02	6
7	Jimmy Neutron: Boy Genius	5.6	2001-12-21	6
8	Race to Witch Mountain	5.5	2009-03-12	6
9	The Empire Strikes Back	8.2	1980-05-17	5
10	Star Wars: Clone Wars: Volume 1	8.0	2005-03-21	5

Figure 10: Successfully working recommendation results - Star Trek, The Fifth Element, Empire Strikes Back

This is the project success screenshot. The recommendation system correctly identified movies similar to Avatar based on genre and keyword matching. Star Trek Into Darkness and The Fifth Element scored 6 points, while The Empire Strikes Back scored 5 points.

Screenshot: Testing with Inception



```
12 CASE WHEN keywords LIKE '%futuristic%' THEN 1 ELSE 0 END +
13 CASE WHEN keywords LIKE '%battle%' THEN 1 ELSE 0 END +
14 CASE WHEN keywords LIKE '%soldier%' THEN 1 ELSE 0 END
15 ) AS similarity_score
16 FROM movies_data
17 WHERE title != 'Inception'
18 HAVING (
19 CASE WHEN genres LIKE '%Action%' THEN 1 ELSE 0 END +
20 CASE WHEN genres LIKE '%Adventure%' THEN 1 ELSE 0 END +
21 CASE WHEN genres LIKE '%Fantasy%' THEN 1 ELSE 0 END +
22 CASE WHEN genres LIKE '%Science Fiction%' THEN 1 ELSE 0 END +
23 CASE WHEN keywords LIKE '%space%' THEN 1 ELSE 0 END +
24 CASE WHEN keywords LIKE '%alien%' THEN 1 ELSE 0 END +
```

Figure 11: Query modified with WHERE title != Inception to test reusability

Completed

Time in queue: 63 ms

Run time: 1.252 sec

Data scanned: 5.43 MB

Results (10)

Copy

Download results CSV

Search rows

< 1 > ⚙

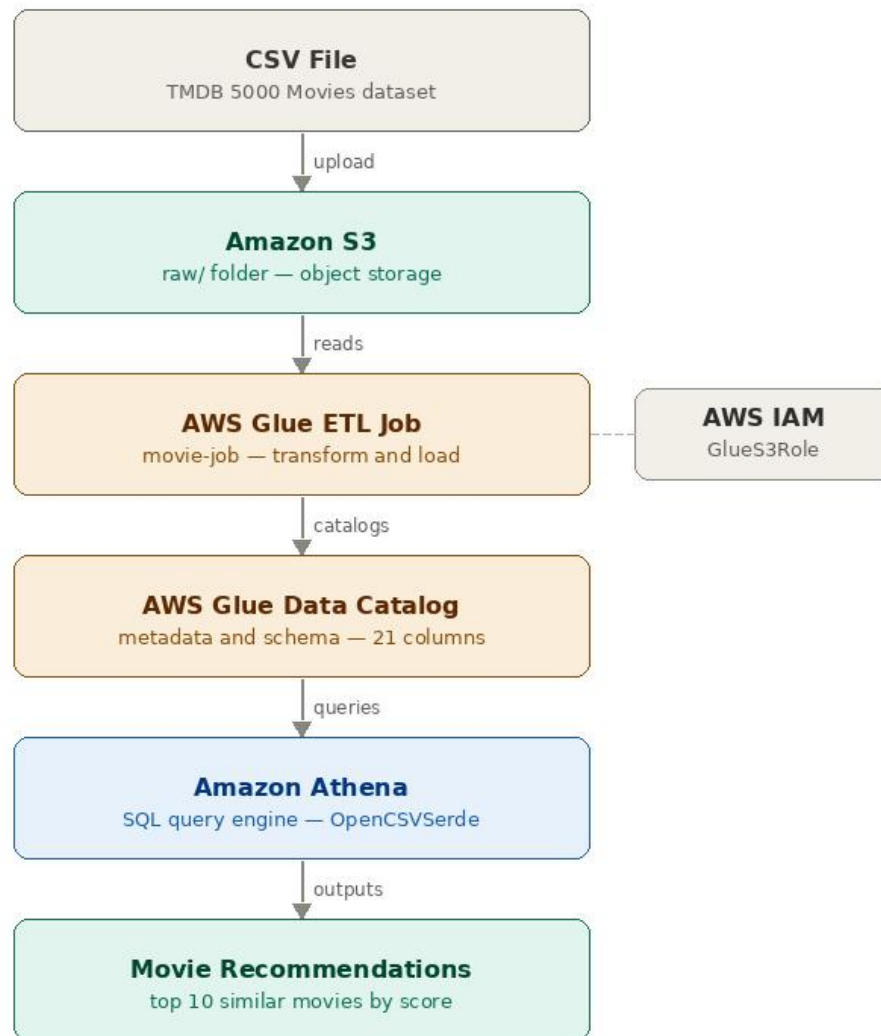
#	title	vote_average	release_date	similarity_score
1	Avatar	7.2	2009-12-10	9
2	Star Trek Into Darkness	7.4	2013-05-05	6
3	The Fifth Element	7.3	1997-05-07	6
4	Independence Day	6.7	1996-06-25	6
5	Starship Troopers	6.7	1997-11-06	6
6	Ender's Game	6.6	2013-10-23	6
7	Final Fantasy: The Spirits Within	5.9	2001-07-02	6
8	Jimmy Neutron: Boy Genius	5.6	2001-12-21	6
9	Race to Witch Mountain	5.5	2009-03-12	6
10	The Empire Strikes Back	8.2	1980-05-17	5

Figure 12: Results showing Avatar at rank 1 with score 9 when Inception keywords were not updated

These screenshots show testing the system with a different movie (Inception). However, since the genre and keyword filters were not updated to match Inception, Avatar scored highest (9 points) because the filters still used Avatar's genres and keywords. This revealed that the genre/keyword list must be updated for each new movie.

4. Architecture Flow

The complete data flow of the project from raw data to recommendations:



5. Final Recommendation Query

```
SELECT
    title,
    vote_average,
    release_date,
    (
        CASE WHEN genres LIKE '%Action%' THEN 1 ELSE 0 END +
        CASE WHEN genres LIKE '%Adventure%' THEN 1 ELSE 0 END +
        CASE WHEN genres LIKE '%Fantasy%' THEN 1 ELSE 0 END +
        CASE WHEN genres LIKE '%Science Fiction%' THEN 1 ELSE 0 END +
        CASE WHEN keywords LIKE '%space%' THEN 1 ELSE 0 END +
        CASE WHEN keywords LIKE '%alien%' THEN 1 ELSE 0 END +
        CASE WHEN keywords LIKE '%futuristic%' THEN 1 ELSE 0 END +
        CASE WHEN keywords LIKE '%battle%' THEN 1 ELSE 0 END +
        CASE WHEN keywords LIKE '%soldier%' THEN 1 ELSE 0 END
    ) AS similarity_score
FROM movies_data
WHERE title != 'Avatar'
HAVING ( ... same expression ... ) >= 2
ORDER BY similarity_score DESC, CAST(vote_average AS DOUBLE) DESC
LIMIT 10;
```

7. Sample Output - Recommendations for Avatar

Rank	Movie Title	Similarity Score	Rating	Release Year
1	Star Trek Into Darkness	6	7.4	2013
2	The Fifth Element	6	7.3	1997
3	Independence Day	6	6.7	1996
4	Starship Troopers	6	6.7	1997
5	Ender's Game	6	6.6	2013
6	Final Fantasy: The Spirits Within	6	5.9	2001
7	Jimmy Neutron: Boy Genius	6	5.6	2001
8	Race to Witch Mountain	6	5.5	2009
9	The Empire Strikes Back	5	8.2	1980
10	Star Wars: Clone Wars: Volume 1	5	8.0	2005